Systems Development Group Project:

Pink Café Case Study

Group B16
Aaron Agoas Antal-Bento - 23013693
Ahmed AlShamy - 24045361
Nickolas Greiner - 24018357
Oliver Churchley - 23020494
Oliver Mercer - 24026901

Introduction

For the Systems Development module, our group selected the Pink Café case study, which requires the development of an application capable of accepting CSV sales data and generating predicted future sales using artificial intelligence techniques. The primary objective of this project is to design and implement an efficient, user-friendly forecasting tool tailored to the needs of a small business environment. The system is intended to support data-driven decision-making by providing accessible and interpretable sales forecasts over specified time periods.

In order to ensure both efficiency and usability, the application was developed as a lightweight web solution. The front-end was implemented using React and Tailwind CSS to create a responsive and intuitive user interface. The back-end was developed in Python, primarily to support integration with Meta's Prophet forecasting algorithm, which is designed for time-series prediction and operates natively within the Python ecosystem. Flask was used to facilitate communication between the front-end and the forecasting model, enabling streamlined server-side processing and deployment. These technology choices were informed by considerations of compatibility, scalability, and maintainability, as well as the team's existing technical expertise.

The project was developed following the Rational Unified Process (RUP), providing a structured and iterative framework for planning, implementation, and evaluation. RUP enabled clear documentation, defined phases of development, and continuous refinement of requirements. Version control was managed using GitHub to coordinate collaborative development and prevent integration conflicts. While RUP requires consistent communication and engagement from all team members, the use of structured documentation and version control tools mitigated potential coordination challenges and supported efficient teamwork throughout the project lifecycle.

Aims and Objectives

The main aim of this project is to develop a functional and efficient application capable of forecasting future café sales using uploaded CSV data. The system is designed to generate clear visual outputs, including graphs and supporting textual insights, in order to enhance user understanding and support informed business decision-making.

To achieve this aim, the following SMART objectives were established:

- Deliver a working web application with a dashboard, sales history, and parameter controls.
- Implement a Prophet AI model achieving ≥ 85% forecast accuracy on held-out sales data.
- Deploy an interactive dashboard fully compliant with WCAG 2.1 AA accessibility standards.
- Complete structured testing to ensure ≥ 95% of test cases pass with no critical defects.
- Develop an interactive data visualisation system displaying predicted café sales of selectable periods through dynamic charts.

These objectives provide measurable milestones to guide the structured development, implementation, and evaluation of the forecasting system.

Literature Review

Introduction

Food waste remains a significant challenge in retail and hospitality, particularly in businesses handling perishable goods such as coffee chains. Inefficient inventory management and inaccurate demand estimation often lead to overproduction, resulting in waste and financial loss. This has increased interest in data-driven approaches that improve forecasting accuracy and support sustainable operations.

Demand forecasting is a key technique, enabling businesses to better align supply with customer demand. Advances in time series analysis and machine learning have introduced methods with varying complexity and accuracy. At the same time, web-based systems allow predictive insights to be delivered in an accessible and actionable format.

This review examines research on food waste reduction and demand forecasting, focusing on forecasting techniques and decision support systems. It also evaluates relevant web technologies to justify the design choices made in this project.

Food Waste & Forecasting Context

Food waste is a major operational and environmental issue in the UK's retail and hospitality sectors, particularly for perishable goods. Inefficiencies in inventory management and unpredictable demand often result in overproduction and unsold stock. Around one quarter of food intended for consumption is wasted globally (Stancu et al., 2016), while UK food waste reached 10.2 million tonnes in 2021–2022 (WRAP, 2025). Globally, over $1 trillion worth of food is discarded annually, contributing 8–10% of greenhouse gas emissions (FAO, 2024).

Demand forecasting is widely recognised as a key strategy for reducing waste. Advances in AI and data analytics have enabled more adaptive decision-making systems (Di Vaio et al., 2020). By analysing historical sales data and identifying patterns such as trends and seasonality, forecasting models improve stock planning and reduce overproduction. Research shows machine learning models outperform traditional methods, improving accuracy and decision-making, which reduces both waste and unmet demand (Rodrigues et al., 2024).

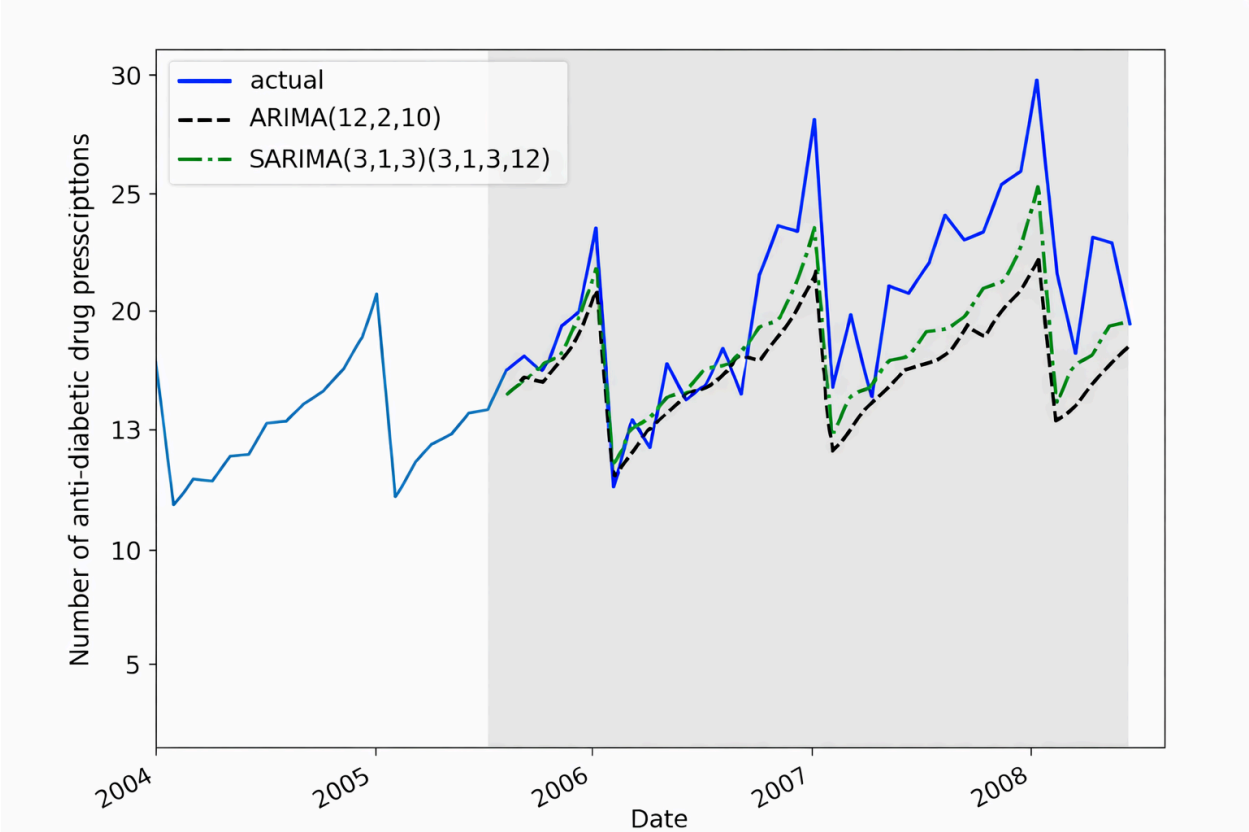
However, forecasting perishable goods is challenging. Limited shelf life means overestimation directly causes waste, while demand is highly variable and influenced by time, seasonality, and external factors such as weather. The FAO estimates that one-third of all food produced globally is lost or wasted (FAO, 2024), highlighting the scale of the problem.

These challenges emphasise the need for accurate, practical forecasting solutions. As a result, businesses increasingly rely on data-driven approaches that adapt to demand variability and provide actionable insights.

Forecasting Techniques

Time series forecasting has traditionally been dominated by statistical approaches, with ARIMA (AutoRegressive Integrated Moving Average) widely recognised as a benchmark model within the literature (Song, 2025). Its popularity is largely due to its strong theoretical foundation and effectiveness when applied to structured, stationary datasets. However, as real-world data has become increasingly complex, several limitations of ARIMA have become apparent. The model requires stationarity, often necessitating preprocessing steps such as differencing, which can increase complexity and risk information loss (Kaur, 2025). Furthermore, ARIMA relies on careful parameter tuning, making it less accessible and more time-consuming to implement effectively.

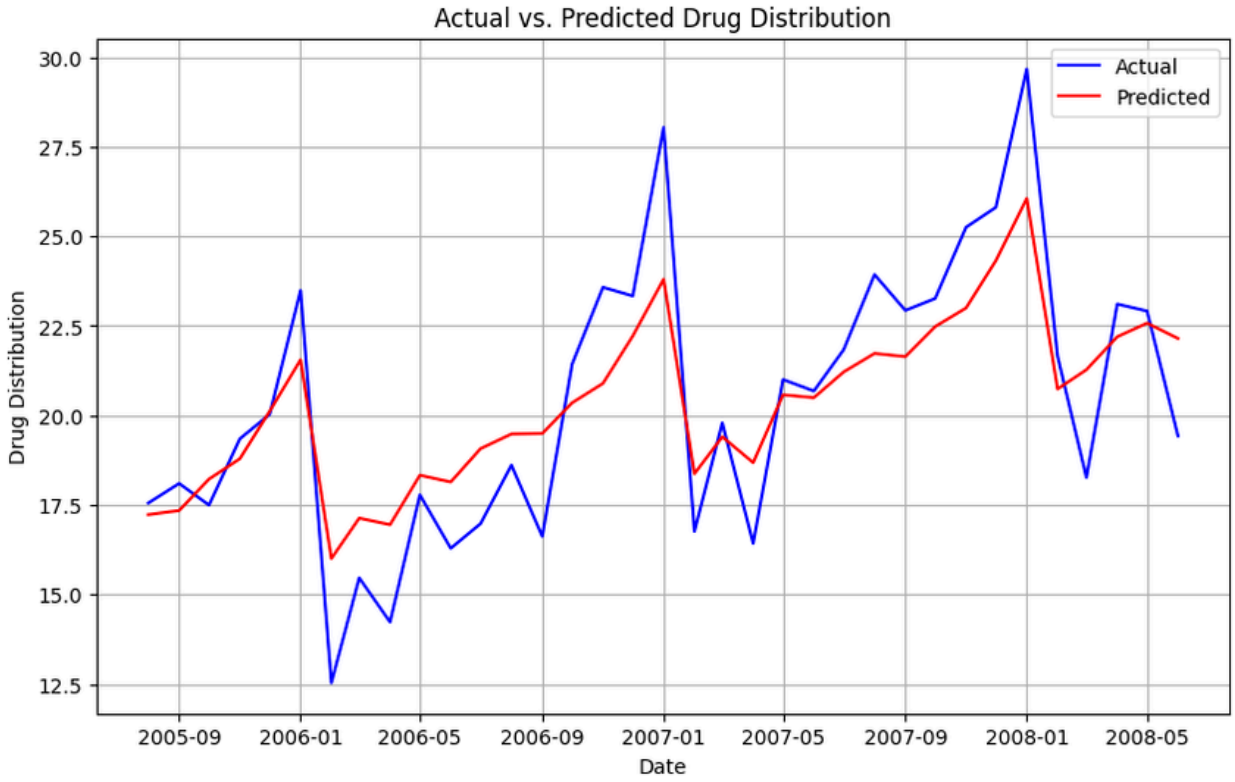
These limitations are reflected in practical applications. As shown in Figure 1, while ARIMA and its seasonal extension (SARIMA) are able to broadly follow the overall trend of the data, they tend to smooth out sharp fluctuations and struggle to accurately capture peaks and sudden changes. This highlights a common issue identified in the literature, where traditional models may underperform when faced with volatile or highly seasonal real-world datasets.



In contrast, machine learning approaches have emerged as flexible alternatives capable of modelling non-linear relationships within time series data. These methods can often achieve higher predictive accuracy, particularly when large volumes of data are available (Song, 2025). However, this improved performance typically comes at the cost of increased computational requirements and reduced interpretability. Many machine learning models also require extensive feature engineering and hyperparameter optimisation, limiting their practicality in real-world business settings (Kaur, 2025).

Prophet represents a more recent approach that attempts to balance the strengths of both traditional and machine learning methods. Developed by Facebook, Prophet is an open-source forecasting tool designed to handle common challenges associated with real-world data (Song, 2025). According to its official documentation, Prophet is based on an additive model that combines non-linear trends with multiple seasonal components, such as daily, weekly, and yearly patterns, alongside holiday effects (Facebook, n.d.). This structure allows it to effectively model complex temporal behaviours without requiring extensive manual configuration.

A key advantage of Prophet is its robustness. It is capable of handling missing data, outliers, and sudden changes in trends, making it particularly suitable for business applications where data quality may be inconsistent (Facebook, n.d.). This is supported by the results shown in Figure 2, where the predicted values closely follow the overall direction of the actual data while maintaining stability across fluctuations. Although some peaks are still smoothed, the model demonstrates a stronger ability to adapt to changing trends compared to traditional approaches



.

When evaluating accuracy versus complexity, the literature highlights a clear trade-off. Machine learning models may offer superior accuracy in highly complex or large-scale datasets but require significant expertise and resources (Song, 2025). ARIMA, while theoretically simpler, can become complex in practice due to preprocessing and tuning requirements and may struggle with long-term forecasting (Kaur, 2025). Prophet, while not always the most accurate, provides a strong balance by delivering reliable performance with relatively low complexity.

In terms of real-world usability, Prophet is often identified as the most practical solution. Its "plug-and-play" nature, minimal preprocessing requirements, and accessibility make it well-suited for rapid deployment in business contexts (Facebook, n.d.). Overall, while no single approach is universally optimal, Prophet offers a balanced and practical alternative that aligns well with the demands of modern time series forecasting tasks.

Decision Support Systems / Web Apps

Decision support models alone are insufficient, as they generate predictions without enabling effective user interaction. In this project, Prophet supports decision-making; however, its value depends on usability and accessibility. Research indicates that user adoption and continued use of applications are strongly influenced by factors such as usefulness and usability, with poor user experience leading to abandonment regardless of functionality (Al-Shamaileh *et al.*, 2023). This demonstrates that accurate outputs alone are not enough, systems must present information in a clear and accessible way to support practical decision-making.

To address this, a web application was selected as the delivery platform. Web applications run directly in browsers without installation, improving accessibility across devices and reducing barriers to use (Yasar,

2024). They also reduce maintenance requirements through automatic updates, ensuring users always access the latest version. In addition, web apps simplify development by enabling a single codebase to operate across multiple platforms, avoiding the complexity and cost of maintaining separate native applications (Amazon Web Services, n.d.).

Although web applications may offer lower performance and limited access to device-specific features compared to native solutions, they provide greater flexibility, scalability, and faster deployment. Overall, web applications represent a cost-effective and practical solution for delivering decision support systems in real-world, multi-user environments.

Tech Stack Justification

The system was implemented using a lightweight and efficient technology stack. The frontend was developed using React, which enables the creation of dynamic and responsive user interfaces through a component-based architecture (React, n.d.), alongside Tailwind CSS for rapid and consistent styling using utility-first design principles (Tailwind Labs, n.d.). The backend was built using Flask, a minimal Python web framework that supports rapid development and straightforward integration with data processing tasks (Pallets Projects, n.d.). SQLite was used as the database due to its simplicity, low overhead, and suitability for small-scale applications without requiring complex configuration (SQLite, n.d.). Together, this stack provides a fast, scalable, and maintainable solution.

Summary

Food waste in perishable goods sectors is driven by inaccurate demand forecasting and inefficient inventory management, creating both economic and environmental challenges. The literature highlights demand forecasting as a key solution, with time series methods enabling businesses to better align supply with demand. Traditional models such as ARIMA offer strong theoretical foundations but struggle with complex, real-world data, while machine learning approaches improve accuracy at the cost of complexity. Prophet provides a balanced alternative, offering robust and practical forecasting with minimal configuration. However, models alone are insufficient without effective delivery. Decision support systems, implemented through web applications, improve accessibility and usability, ensuring insights can inform real decisions. Combined with a lightweight technology stack, this project delivers a scalable, practical solution for reducing waste through data-driven forecasting.

Requirements

The system requirements define the needs and expectations of users and are categorised into functional and non-functional requirements. Functional requirements describe what the system must do, while non-functional requirements define how the system should perform. These requirements were identified based on the needs of the Pink Café case study, with a focus on usability, performance, and reliability.

The requirements were reviewed within the group to ensure they were clear, feasible, and aligned with the project objectives. They were then prioritised so that core functionality, such as data processing and prediction generation, was implemented before additional features. As

development progressed, the requirements were refined to reflect a clearer understanding of system needs and technical constraints.

Functional requirements:

- The system must provide a frontend interface that allows users to interact with and view sales data.
- The system must process uploaded .csv files to generate sales predictions using a forecasting algorithm.
- The system must display predicted sales data using graphs with selectable products and timeframes.
- The system must adapt its training period to the size of the dataset in order to achieve the highest possible accuracy.
- The system must authenticate users through a login system before granting access.
- The system must provide configurable settings that allow users to adjust model parameters.

Non-functional requirements:

- The system must be responsive and function correctly across mobile, tablet, and desktop screen sizes.
- The interface must maintain a consistent design (colour scheme, typography, layout).
- Graphs must include clear labels, titles, and legends and render without visual errors.
- The system must present output data in a structured and readable format.
- The system must implement security measures such as password hashing and restricted access to protected pages.
- The system must generate predictions within an acceptable time frame.

Clearly defined requirements provided a structured foundation for development, ensuring all team members understood the system's objectives and expected functionality. This helped keep the project on track by guiding implementation decisions, reducing ambiguity, and allowing progress to be measured against agreed criteria.

Project Planning & Team Roles

Project Planning

Structured planning was essential to ensure the project was delivered within the specified deadlines. The team adopted the Rational Unified Process (RUP), which provided a structured yet iterative framework for managing the development process. RUP organises development into defined phases while still supporting iterative refinement of system components. This allowed the team to plan tasks clearly, allocate responsibilities, and regularly review progress throughout the project. As a result, the approach supported the gradual development and improvement of system components while ensuring the project remained aligned with the defined requirements and overall objectives.

The project was divided into five key phases:

1. **Requirements Analysis** – Interpreting the project brief and defining aims, objectives, functional and non-functional requirements.

2. **System Design** – Designing the frontend layout, database structure, and prediction model architecture.

3. **Implementation** – Developing the user interface (front-end), back-end, and sales prediction algorithm.

4. **Testing & Validation** – Evaluating algorithm performance, verifying prediction outputs, and testing the application.

5. **Documentation & Evaluation** – Preparing the final report, reflecting on system limitations, and suggesting improvements.

Communication was managed through a dedicated Discord channel, where shared lists were used to allocate responsibilities, track progress and discuss issues. Tasks were broken down into smaller, more manageable components to ensure clarity and distribute the workload evenly across the team. Bi-weekly meetings were held to review progress, address technical challenges, and define clear objectives for the upcoming week.

Version control was managed using Git to allow collaborative development without overwriting each other's work. This also enabled traceability of changes and reduced integration issues when combining frontend and backend components. Each merge to the main branch also required approval from at least two team members which helped keep the code clean and consistent, as well as ensuring all changes remained aligned with the project's requirements.

Team Roles

Rather than assigning fixed roles for the duration of the project, responsibilities were rotated throughout the development lifecycle. This approach ensured balanced contribution and allowed all team members to work on multiple aspects of the project. While responsibilities were clearly defined at each stage, ownership of specific roles shifted across iterations to promote shared accountability and a broader understanding of the overall system architecture.

Project Coordinator

This role took responsibility for assigning tasks, monitoring deadlines, and ensuring that all components aligned with the project requirements.This also involved maintaining consistency in the codebase and file structure, as well as addressing any technical or organisational issues that arose.

Back-end & Algorithm Developer

This role focused on implementing and evaluating the sales prediction model, as well as developing the application's backend API. Key responsibilities included:

- Implementing the backend API to communicate with the frontend
- Model selection and training
- Evaluating model performance and accuracy
- Generating prediction outputs to use in the front-end

Front-end Developer

This role was responsible for designing and implementing the graphical user interface. Key tasks included:

- Creating a user-friendly interface
- Creating responsive layouts
- Integrating graphical visualisations
- Ensuring usability and consistent styling

Security & Testing Lead

This role focused on implementing the login system and ensuring basic data protection measures were in place. Additionally, this role conducted system testing and evaluation.

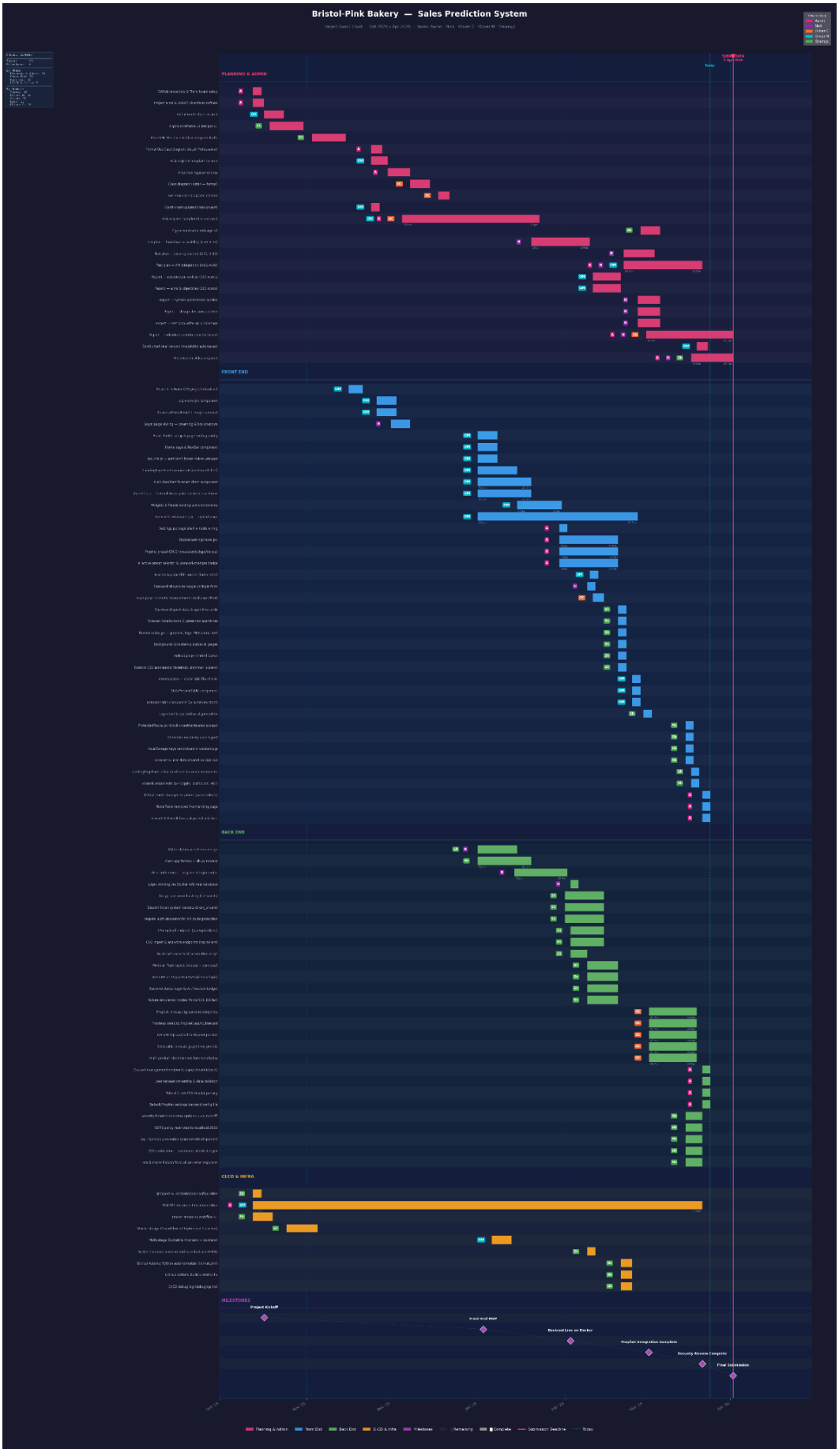
Documentation & Design Specialist

This role focused on producing clear and accurate design diagrams, visualisations, and project documentation, providing both a visual and written record of the project to effectively communicate technical concepts.

Collaboration Approach

Although responsibilities were divided, all major decisions were made collaboratively. Peer review within the group ensured quality control and helped identify issues early. This structure allowed us to simulate a professional development workflow while remaining realistic about the constraints of a university project.

We used a Gantt chart to plan the project timeline, allocate tasks, and ensure that key milestones and deadlines were met. This was updated throughout the project to adapt to progress and setbacks.


Bristol-Pink Bakery — Sales Prediction System
PLANNING & ADMIN
FRONT END
BACK END
CI/CD & INFRA
MILESTONES
Project Kickoff
Front End Dept
Backend Live on Docker
Prophet Integration Complete
Security Review Complete
Final Submission
Today
Planning & Admin
Front End
Back End
CI/CD & Infra
Milestones
Remaining
Complete
Submission Deadline
Today

Risk management was also considered, and a risk register was created to identify and mitigate potential risks. Updates were made as new risks were identified throughout the project. Overall, the risk register improved the team's awareness of potential issues and supported more proactive management.

ID	Risk Description	Likelihood	Impact	Mitigation / Prevention	Contingency Plan
R1	Team member(s) missing workshops or meetings, causing delays in planning and development.	Medium	Medium	Agree clear attendance expectations and maintain shared communication channels	Reassign tasks temporarily and adjust timeline
R2	Requirements misunderstood or incorrectly validated with tutor	Medium	High	Regular check-ins with tutor and team discussions about what the requirements are	Re-scope features, prioritise essential requirements first
R3	Skills gaps in the group slowing implementation	Low	Low	Early skills assessment using CVs and allocate tasks based on strengths	Reallocate tasks and simplify features if needed
R4	Version control issues (merge conflicts, lost work)	Medium	High	Use Git branches for each feature and commit regularly with clear messages	Revert to stable branch
R5	Integration problems between different modules of the system	Medium	Medium	Set consistent coding standards and run integration tests early	Delay advanced features and merge minimal working version
R6	Conflicts or communication breakdown within the team	Low	Medium	Rotate roles and resolve issues quickly in workshops	Escalate to tutor for mediation
R7	Major issues discovered late during testing	Low	High	Test regularly and after each new feature addition, fix issues immediately to avoid build up	Prioritise defects based on severity and temporarily pause new feature development
R8	Presentation not running smoothly	Low	High	Rehearse full demo multiple times, and ensure all group members understand all the features	Let another group member take over to keep the delivery clear and structured

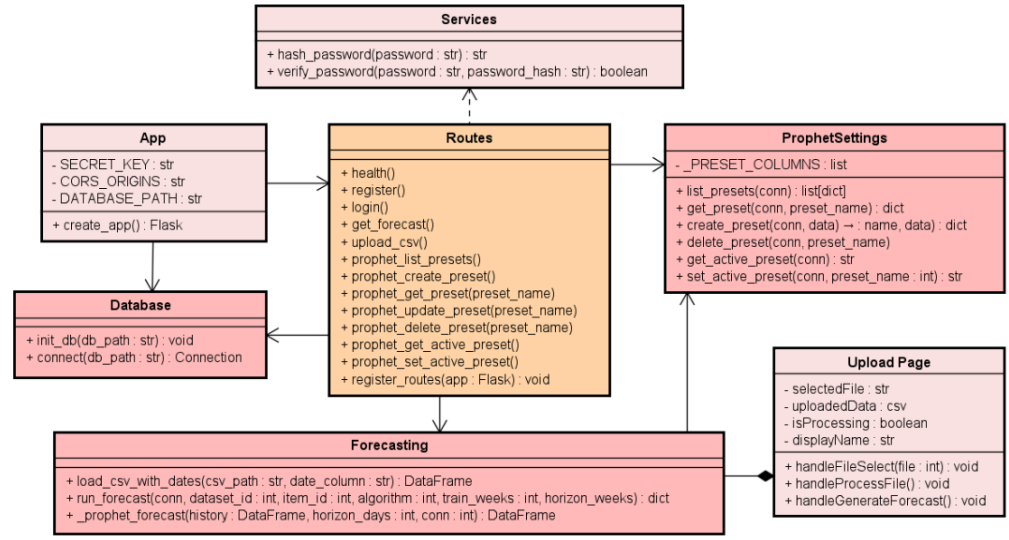
---

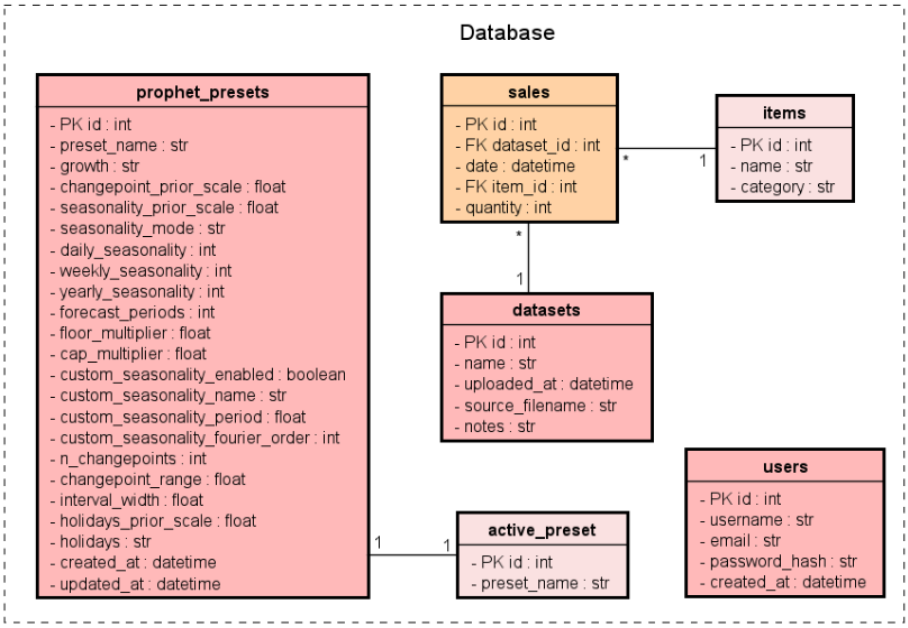
Design and Test Plan

System Architecture

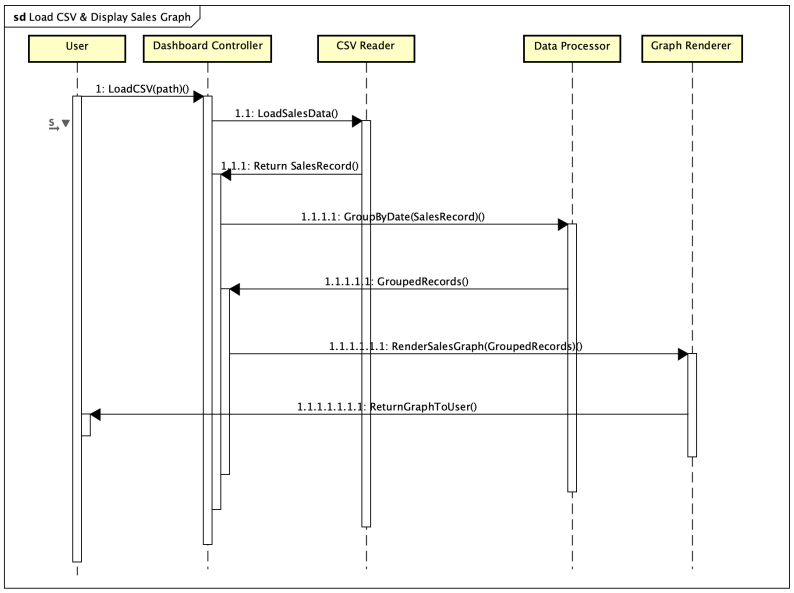
The system is implemented as a web-based client–server application, it consists of a frontend user interface, backend application logic, and a relational database. The frontend manages user interaction and presentation, while the backend processes requests, performs validation, and communicates with the database. Persistent data such as user accounts, application settings and uploaded datasets are stored in the database. Communication between components occurs via HTTP requests, ensuring separation of concerns and scalability.

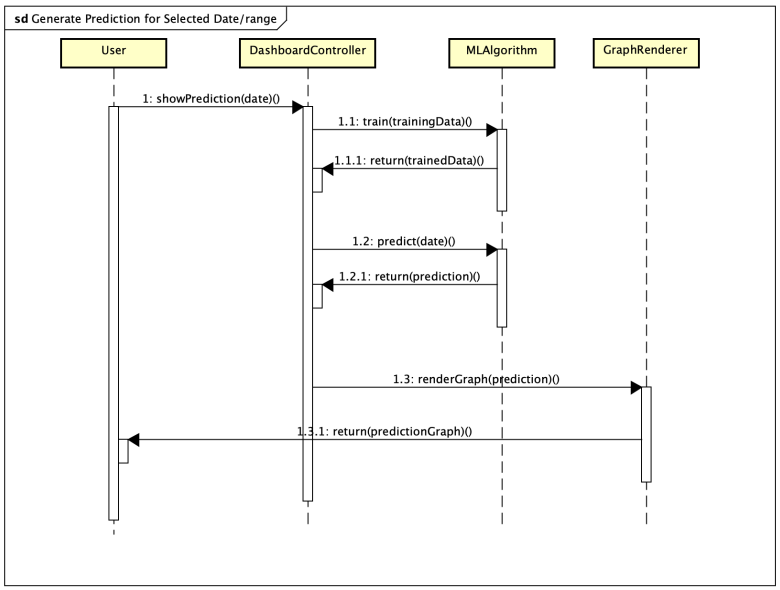
A class diagram was created to help us maintain consistency throughout development and to reduce integration issues by giving the team a structure to work from:





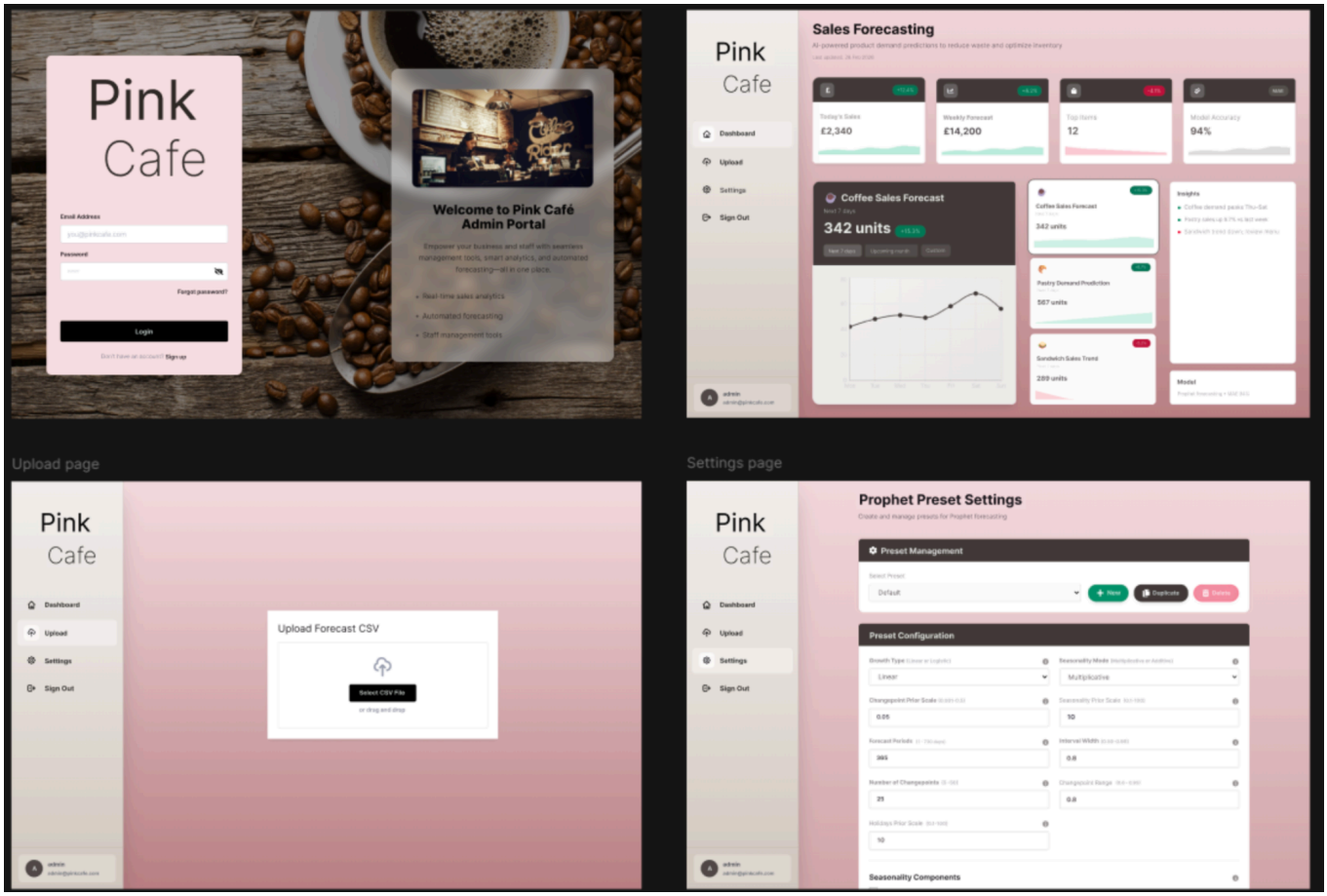
The sequence diagrams helped us to visualise interactions between components:



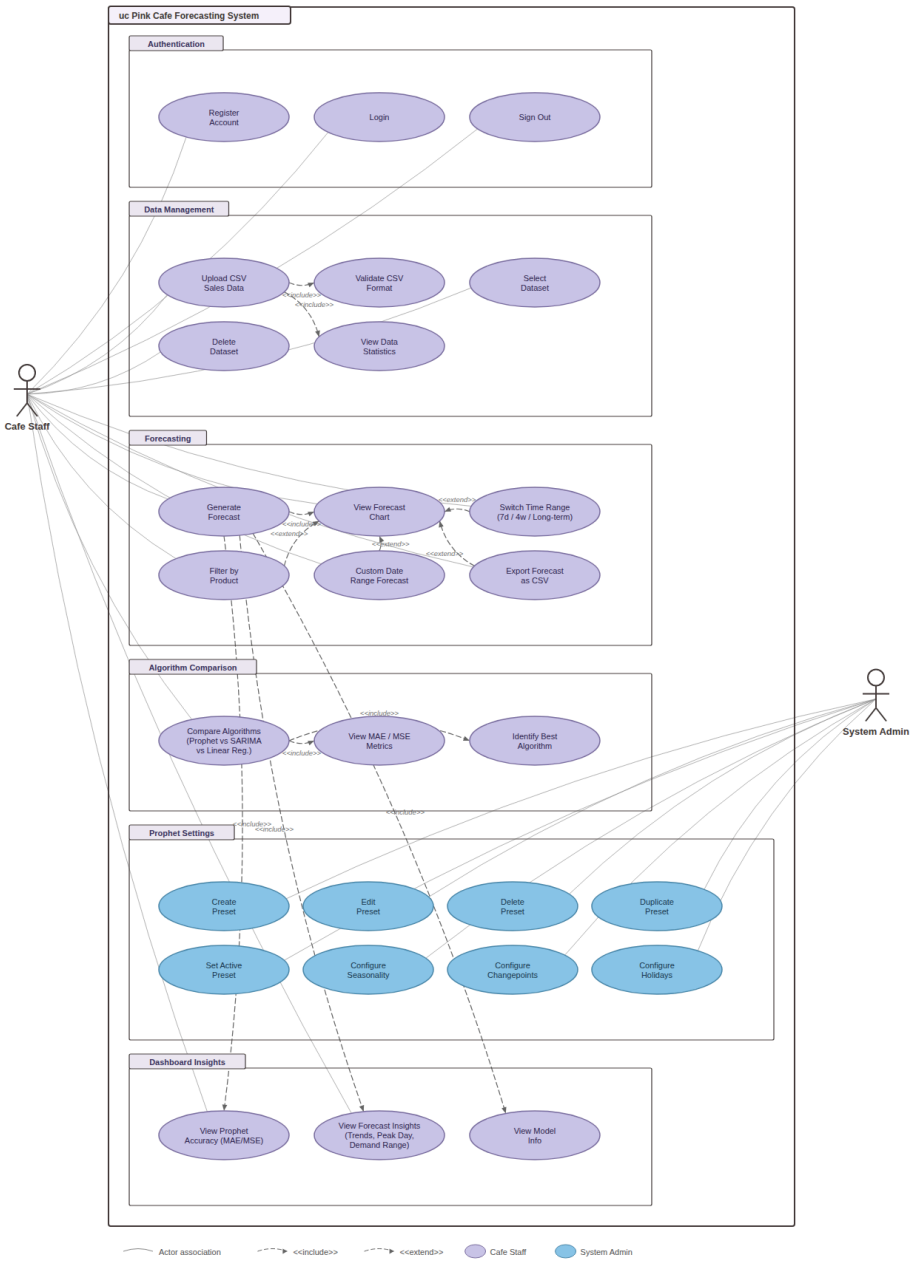


User Interface Design

The user interface was designed in Figma prior to implementation to define page layout, navigation flow, and input layout. The design emphasises clarity and usability through consistent pages, navigation menus, and logical grouping of controls. Key screens including the login page, main dashboard, and settings page are shown in the following images.



The use case diagram helped us to define system functionality from the user's perspective, ensuring requirements were clearly understood and consistently implemented:



Design Decisions

Several design decisions were made to ensure maintainability and readability. The system is split into interface, logic, and data parts so each can be developed and tested separately. A relational database was selected to enforce structured data storage and integrity constraints. Authentication is handled server-side to protect sensitive data and prevent unauthorised access. These decisions improve scalability, security, and ease of future extension.

Testing Strategy

A structured testing strategy was applied to verify that the system meets functional and usability requirements. Functional testing validated core features such as authentication, navigation, and data processing. Usability testing ensured interface elements render correctly and user workflows operate as expected. Database tests confirmed correct data creation, retrieval, update, and deletion. Tests were executed manually in a controlled environment using a local deployment. Each functional requirement was mapped to at least one test case to ensure coverage. We also did a security testing strategy where we tested things like, error handling, password encryption and session management to ensure our application was secure for every and all users.

Test Plan

The following test plan was created to validate functionality and usability across the application.

Test Plan - Function and Usability

ID	Test Description	Test Procedure	Expected Result	Actual Result	Action Taken
F-01	Application launch	Open application via localhost	Application loads successfully	Application successfully opened on http://localhost:3000	N/A
F-02	Login functionality	Submit valid credentials	User is logged in and redirected to dashboard	Submitted username and password but was not redirected	Updated the Dockerfile to enable credential validation in database
F-03	Forecast graph rendering	Generate prediction	Graphs display correctly	Tried to generate predictions but graph didn't render	Implemented Prophet model and connected the correct api endpoint
F-04	Data accessible and stored in the database.	Load CSV data from frontend	Data uploads correctly and is available from the database.	CSVs are correctly stored in the database and are accessible in the application.	N/A
F-05	Navbar responsiveness	Resize screen and navigate	Nav bar is responsive on different screen sizes and all buttons link correctly	Layout adapts and links work	Fixed spacing issues on some screen sizes
F-06	Settings persistence	Create and save forecast settings	Settings changes are saved in the database and are applied in subsequent forecasts	New training presets are saved in the database and the selected settings are used	N/A
F-07	Dynamic graph updates	Modify dataset and reload	Graph updates accordingly	Graphs correctly update following data changes	N/A
F-08	Database persistence	Restart application	Data remains stored	Settings and datasets remain saved after the application is restarted.	N/A
F-09	CSV upload	Upload dataset	File is processed and confirmed	File is successfully uploaded and processed and user receives confirmation.	N/A

F-10	Prediction period selection	Select different timeframes	User can select multiple different prediction periods (between 4 - 8 weeks)	Prediction period shows correct predictions for the selected time frame.	N/A

Test Plan - Security

ID	Test Description	Test Procedure	Expected Result	Actual Result	Action Taken
S-01	Password security	Inspect database values	Passwords are hashed	Users passwords in database are unreadable as plain text	N/A
S-02	API protection	Call API without token	Access denied	Failed, access granted to private endpoints	Added authentication decorators in backend stopping unauthorised access
S-03	Route protection	Directly enter the URL of a protected page	Block access and redirect to login	Failed, user can access any page without logging in	Added security checks that block access unless signed in
S-04	Session persistence	Refresh/close and reopen browser	Session maintained	Passed, but users are still taken to login page even when signed in	Changed default route so logged in users get taken straight to the home page
S-05	Invalid login handling	Submit incorrect credentials	Error message displayed	Passed, user was not logged in and an error message was shown	N/A
S-06	Duplicate registration	Register an account with an email that already exists	Error returned	Passed, Correct error returned: "Username or email already exists"	N/A

Test Plan - Performance & Stress

ID	Test Description	Test Procedure	Expected Result	Actual Result
P-01	Forecast response (4 weeks)	Time repeated API calls on the coffee dataset	Acceptable response time	cold = 291.94 ms, average = 194.11 ms
P-02	Forecast response (8 weeks)	Time repeated API calls on the coffee dataset	Acceptable response time	cold = 270.71 ms, average = 279.93 ms
P-03	CSV processing performance	Time uploading and processing the coffee dataset	Efficient processing time	167.71 ms, 230 rows, 2 products, noMissingValues=True, chronological=True
P-04	Multi-item analytics	Timed multi-item analytics by running forecast requests for all items	Scales efficiently	1116.13 ms total workload; Americano: 267.81 ms, 56 forecast points, total_predicted_units=3555.69 Cappuccino: 267.82 ms, 56 forecast points, total_predicted_units=6700.02

Test Plan - UI / Accessibility

ID	Test Description	Test Procedure	Expected Result	Actual Result
U-01	Responsive layout	Test multiple screen sizes	Layout adapts correctly	Page correctly conforms to the size of the display
U-02	Colour contrast	Inspect UI readability	Text clearly visible	Everything can be easily read, with no contrast issues
U-03	Font consistency	Check typography	Consistent open-source fonts	Fonts are consistent, and are all open source

---

Implementation and Testing

Implementation

Following industry standards, and after much team discussion, we managed to decide on the technology stack that we went on to leave unchanged and utilised throughout the development of our application. This tech stack includes using React & Tailwind to handle the frontend development of the app, Python Flask to handle database communication, SQLite to keep database management easier by storing it as just a .db file, and finally using Meta's Prophet forecasting algorithm for the main function of the app. As a team, we had experience in almost all of these technologies and choices, which resulted in the development moving fairly smoothly without much initial learning needed. To accompany this tech stack, we also utilised GitHub for its version control capabilities, alongside docker to ensure we had no problems getting the virtual environment to work across all of our systems.

React and tailwind was an easy decision for our frontend development, as it is used widely throughout the industry and we had some team knowledge going into the project. R&T makes integration incredibly easy as it allows for a more modular approach to frontend development, and combines javascript with html within the same file. This modular format also makes development as a team far easier, as components can be separately worked on and pulled together within one file. This decomposition into components makes using react and tailwind in unison incredibly scalable, as it means the project can grow as needed, and still maintain an easy to understand and develop nature.

Flask was utilised for the backend development, and this was due to all members of the team having used it in the past, as well as our chosen algorithm - meta's Prophet, being developed in python. This made backend development far simpler as both the algorithm and our database logic could be implemented in the same language. Python Flask was chosen due to its immense scalability, as python has many libraries and imports that can be used to scale database logic to almost any level, for example importing sha256 for encryption of sensitive data. We decided on using SQlite for its modularity, as it is simply a .db file, which allowed us to share a database without each individually running a database i.e through mySQL. This made development much more consistent between us all and helped when sharing development across the team.

For algorithms there were a few good examples, but we settled on Meta's Prophet algorithm, as it is very scalable, so it works well with our small data set, but could also be used with much larger datasets. It is also fairly efficient and quick to run, which was necessary as the end user would not want to wait for the algorithm to create predictions, and this meant that everything runs smoother and much faster. Prophet also has lots of modularity and settings, which allow it to be completely altered to suit the users needs, so this made it ideal going into the project, as the end user can alter it in many ways to suit the predictions they need. This includes seasonality and country-specific holidays, making it much more accurate than other algorithms.

To accompany this tech stack, GitHub and Docker were used throughout development, which allowed us to have version control and also to collaborate far easier by just pulling the latest version of development to our own local systems. Docker helped resolve any issues we would have with having our own personal virtual environments, and maintained consistency across development with consistent containerisation - making integration far easier. GitHub allowed us to handle the immense scalability of our project with version control, as well as modularity through separate branches for different components, before pulling the project back together into main with a final pull request.

Overall this tech stack and selection of components really helped to boost the development and management of our project, ensuring we were all up to date as well as keeping up with the scalability of the project. It also meant our model pipeline was fairly straight-forward and easy to keep up with, ensuring efficient and consistent development and the desired output being met.

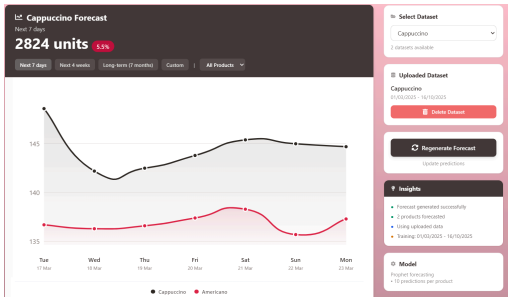
Testing

A combination of functional, integration, system, usability, and security testing was applied to ensure thorough validation of frontend behaviour, backend processes, model predictions, and data handling. Functional testing verified that individual features met their requirements, while integration and system testing ensured that interactions between components and complete end-to-end workflows operated correctly. Usability testing assessed the responsiveness and overall user experience of the interface, and security testing verified the protection of sensitive data such as user credentials. Testing was conducted iteratively throughout development, with failures triggering immediate fixes followed by regression testing to ensure stability. Tests and their results are presented in the test plan above, and this section provides supporting evidence. Below are the test plan IDs alongside evidence, such as screenshots and results, confirming that the tests were successful.

Functional & Usability Tests

F-03 Home dashboard renders Prophet forecast graphs.

Evidence showing generated prediction graph with coffee sales (Cappuccino, Americano).



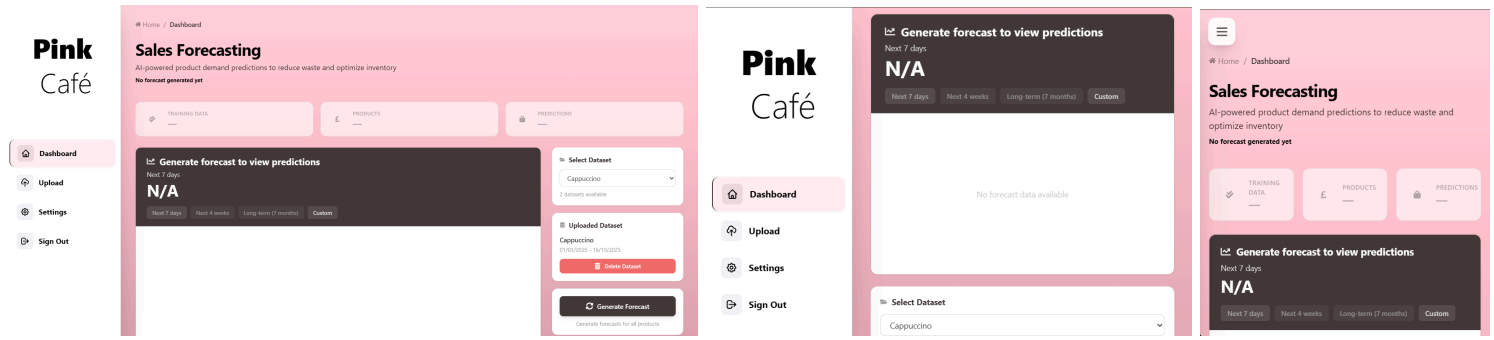
F-04 Database connection and CSV data accessible from the frontend.

Evidence showing database storing uploaded datasets.

	id	name	uploaded_at	source_filename
1	1	Pink_MilkshakeSales_March_-_Oct_2025.csv	2026-03-17 17:...	Pink MilkshakeSales March - Oct 2025.csv
2	3	Pink_CoffeeSales_March_-_Oct_2025.csv	2026-03-17 17:...	Pink CoffeeSales March - Oct 2025.csv

F-05 Responsive navbar and navigation links.

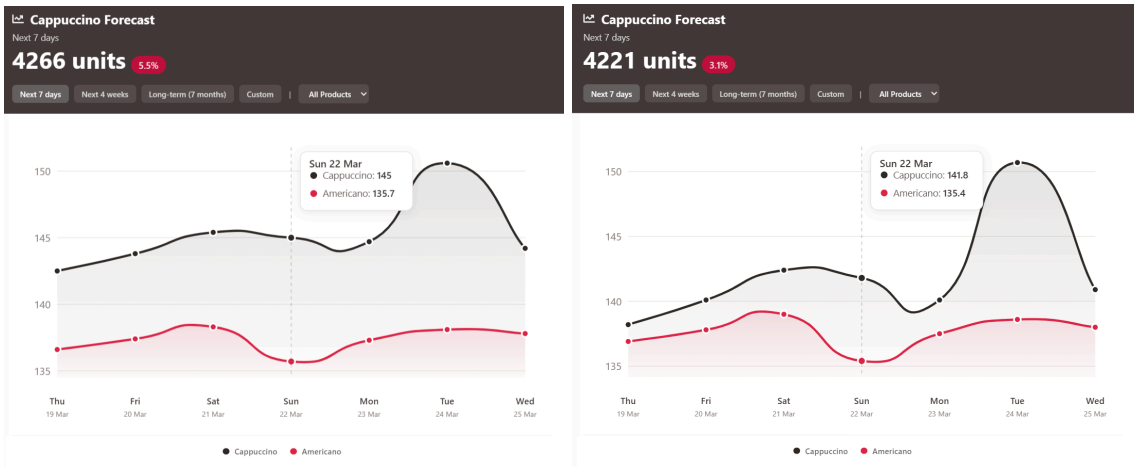
Evidence showing responsive web design for navbar on 3 different screen sizes.



F-06 User-configurable forecast parameters on settings page.

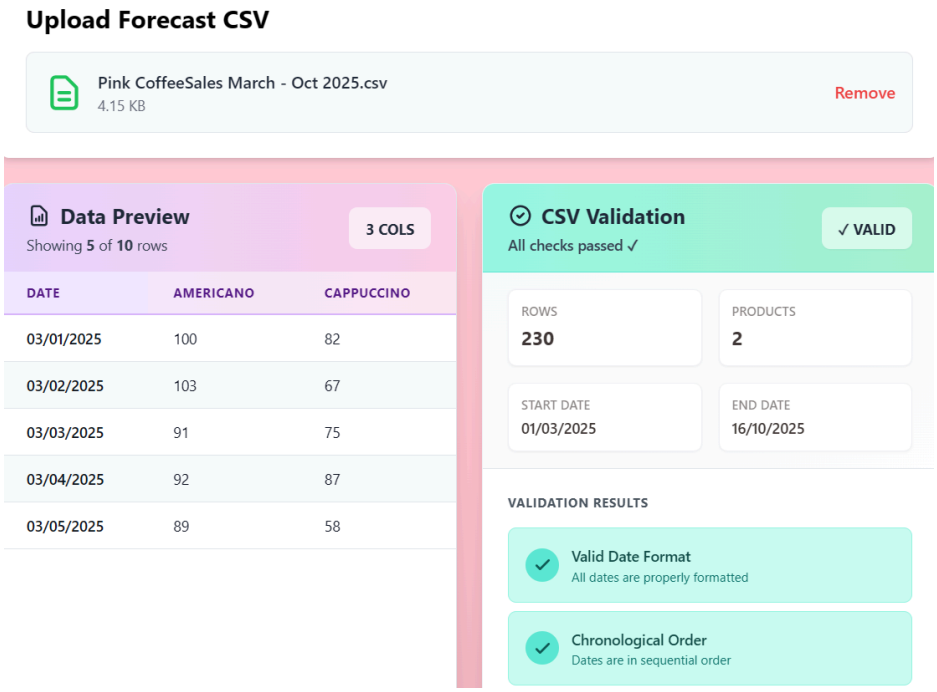
Evidence showing a new preset saved in the database along with prediction graphs for the same data with different training presets.

	preset_n...	growth	changep...	seasonali...	seasonali...
1	Default	linear	0.15	15	additive
2	new preset	linear	0.15	15	multiplicative
3					



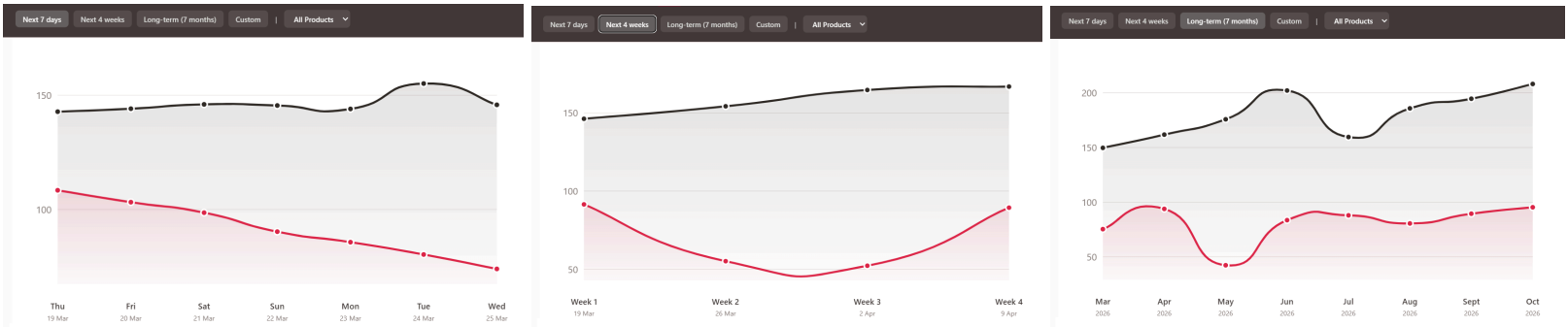
F-09 File selection and ingestion on CSV upload page.

Evidence showing successfully uploaded dataset with preview information and statistics.



F-10 Prediction period selector.

Evidence showing prediction graphs for 7 days, 4 weeks, and 7 months.



Security Tests

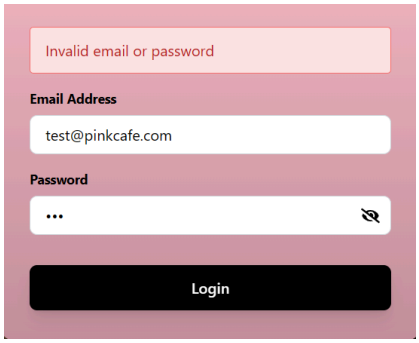
S-02 API route protection.

Evidence showing error message return when trying to access an api route with no authentication.



S-05 Invalid login handeling.

Evidence showing incorrect login and error message.



Performance & Stress Tests

P-01 Prophet forecast response time (4 week).
Results - P01 cold = 291.94 ms, P01 average = 194.11 ms

P-02 Prophet forecast response time (8 week).
Results - P02 cold = 270.71 ms, P02 average = 279.93 ms

P-03 Large CSV upload and processing.
Results - 167.71 ms, 230 rows, 2 products, noMissingValues=True, chronological=True

Evidence showing output json with testing data for tests P01, P02, and P03.


```
{
    "timestamp":  "20260324-195024",
    "dataset_id":  7,
    "item_tested":  "Americano",
    "P01_cold_ms":  291.94,
    "P01_avg_ms":  194.11,
    "P02_cold_ms":  270.71,
    "P02_avg_ms":  279.93,
    "P03_upload_ms":  167.71,
    "P03_rows":  230,
    "P03_products":  2
}
```


P-04 Analytics on multi-item dataset.
Results - 1116.13 ms total workload;
Americano: 267.81 ms, 56 forecast points, total_predicted_units=3555.69
Cappuccino: 267.82 ms, 56 forecast points, total_predicted_units=6700.02

Evidence showing output JSON for multi-item analytics with product-by-product breakdown.


```
[
    {
        "item":  "Americano",
        "elapsed_ms":  267.81,
        "forecast_points":  56,
        "total_predicted_units":  3555.69
    },
    {
        "item":  "Cappuccino",
        "elapsed_ms":  267.82,
        "forecast_points":  56,
        "total_predicted_units":  6700.02
    }
]
```

Evaluation & Lessons Learned

System Performance Evaluation

The developed bakery sales prediction system successfully met its core functional objectives by processing CSV data, identifying key products, and generating future sales predictions using our prophet model. The system was able to extract and analyse historical sales data efficiently, allowing the identification of the top three selling food items and coffee products, and drinks across the selected cafés. This gave a solid starting point for making predictions later on.

When it came to predictions, the system estimated sales for the next 7 days or next four weeks as well as several months, and these mostly followed the patterns seen in the past data. For example, products with more steady demand, like popular coffees, had predictions that stayed quite consistent and realistic. This shows the system was able to pick up on general patterns in the data. However, the accuracy wasn't the same for every product, as items with more unpredictable sales were harder to forecast. Also, things like holidays caused noticeable changes in sales, which the system didn't always fully account for, making some predictions less reliable.

The dashboard component of the system also performed effectively in presenting both historical and predicted data in a clear and structured format. Graphical visualisations allowed users to easily interpret sales patterns over time, while features such as period selection and adjustable training data enhanced the system's flexibility. This ensured that users could explore different scenarios and gain deeper insights into sales behaviour.

Overall, the system achieved its primary goal of supporting data-driven decision-making for stock planning and waste reduction. While there are limitations in predictive accuracy, particularly for variable products, the system provides a functional and practical tool that demonstrates the potential of machine learning in a real-world business context.

Prediction Accuracy & Data Limitations

The accuracy of the predictions produced by the system was mixed and depended largely on the size and quality of the csv dataset provided. One of the main challenges was that the dataset available for this project was limited, which made it more difficult for the model to learn reliable patterns. As a result, the predictions were able to capture general trends but struggled to reflect more complex or irregular sales behaviour.

For products with consistent demand, such as in our synthetic milkshakes dataset, the predictions were generally more accurate. These products tend to follow steady sales patterns, which made them easier for the model to recognise and forecast. However, products with more unpredictable demand, such as croissants, were harder to predict. Sales for these items can change more frequently depending on factors such as customer preferences or daily demand, which the model could not always capture accurately with the limited training data.

Model/Algorithm Effectiveness

The effectiveness of the algorithms used in this project varied depending on the type of data and the complexity of the sales patterns. Simpler models, such as Linear Regression, performed reasonably well when identifying general trends in the data, particularly for products with stable demand. However, the use of simple models also introduced limitations, as Linear Regression assumes a consistent relationship within the data, which is not always realistic in a real-world bakery setting where sales can fluctuate significantly. As a result, the model struggled to accurately predict sudden changes in demand, such as spikes caused by busy days or holidays. More complex models were considered, but due to the limited size of the dataset, they were not always suitable, as they require larger amounts of data and risk overfitting. Overall, the chosen Prophet model provided a good balance between simplicity and performance, demonstrating the ability to generate useful predictions despite some limitations.

Dashboard & User Experience Evaluation

The dashboard was designed with a strong focus on usability and visual appeal, with particular effort put into making it as user-friendly as possible. Care was taken to ensure that the interface was simple to navigate, allowing users to quickly access key features such as viewing sales trends, and adjusting the training period and easily adding your own csv dataset. This made the system accessible even for users with limited technical knowledge. However, only admins should access this app.

A modern front-end design was implemented to improve the overall user experience. This included the use of smooth animations, particularly in loading screens and graph transitions, which helped make the system feel more responsive and engaging. These elements contributed to a more professional and polished appearance, improving the overall quality of the dashboard.

The visual design also followed a consistent pink colour scheme to match the branding of “Pink Cafe.” This created a clear and cohesive identity for the system, making it visually distinctive while also aligning with the business context. The graphs themselves were designed to be clear and easy to interpret, allowing users to quickly understand both historical and predicted sales data.

Overall, the dashboard provided an effective balance between functionality and design. While there is always room for further refinement, the system successfully delivered a clean, modern, and user-friendly interface that supports practical decision-making.

Risk Outcome Evaluation

Several risks identified during the planning phase did occur during the development of the project. One of these risks was time constraints, which affected the level of experimentation that could be carried out with different algorithms. While the system was successfully completed, there was limited opportunity to test a wider range of models or fully optimise the chosen approach. This meant that some potential improvements to prediction accuracy could not be explored.

Overall, while some risks did impact the project, they were managed well enough to ensure a functional final system. However, the experience shows that having more time and a larger dataset would significantly reduce these risks and improve the overall quality of the solution.

Key Successes

One of the key successes of this project was the successful development of a fully functional end-to-end system that integrates data processing, predictive modelling, and an interactive dashboard. The system was able to take raw CSV sales data, transform it into graphs that correctly shows future predictions, demonstrating a strong application of our Prophet implementation within a real-world business scenario. This highlights the effectiveness of the overall system design and implementation.

Another achievement was the usability and visual quality of the dashboard. Significant effort was put into creating a modern, user-friendly interface, which resulted in a clean and intuitive experience. The inclusion of interactive graphs, adjustable training periods, and smooth animations enhanced both functionality and engagement, making the system accessible to non-technical users while still offering useful analytical depth.

The project also benefited from continuous improvement through feedback. Input from peers and mentors was used to refine both the system and its presentation, leading to better design decisions and clearer functionality. This iterative approach ensured that issues were identified early and improvements were consistently applied.

Overall, the project successfully met its objectives and demonstrates a strong foundation for further development into a more advanced and commercially viable solution.

Limitations & Weaknesses

Despite the system's success, several limitations were identified. The most significant was the limited dataset size. Although the training period made use of the full data, it was still insufficient for capturing complex sales patterns, reducing prediction accuracy, especially for products with irregular demand.

Another key weakness was the absence of external factors. The model relied only on historical sales data and did not include variables such as weather, holidays, or local events, all of which can strongly influence customer behaviour. This made it difficult for the system to accurately predict sudden changes in demand.

Additionally, there was limited experimentation with alternative algorithms due to time constraints. While the chosen model performed adequately, further optimisation and comparison with more advanced approaches could have improved results.

Lessons Learned

This project highlighted the importance of data quality and quantity in machine learning. The limited dataset reduced prediction accuracy, showing that reliable models depend heavily on sufficient and varied data. It also demonstrated that simpler algorithms can be more effective than complex ones when working with smaller datasets, reinforcing the need to match model choice to available data.

Another key lesson was the value of iterative development. Feedback from peers and mentors helped improve both the system and its presentation. Finally, time management proved critical, as limited time restricted testing and optimisation, emphasising the need for better planning in future projects.

Future Improvements

There are several ways the system could be improved in future development. Incorporating external factors such as weather conditions, holidays, and local events would allow the model to better reflect real-world influences on customer demand, improving prediction reliability.

Additionally, experimenting with more advanced machine learning models and optimisation techniques could enhance forecasting performance and better capture complex patterns. The dashboard could also be expanded to support multiple user roles instead of admin-only access, increasing usability in a business setting. Finally, deploying the system as a fully functional web application would improve accessibility and make it more practical for real-world use.

References

WRAP(2025) *Food Waste and Food Surplus in the UK - Key Facts July 2025*[online]. Banbury: WRAP. Available from: https://www.wrap.ngo/resources/report/uk-food-waste-food-surplus-key-facts#download-file [Accessed 17/03/2026]

FAO(2024) *Food Waste Index Report 2024. Think Eat Save: Tracking Progress to Halve Global Food Waste*[online]. Rome: FAO. Available from: https://wedocs.unep.org/items/dbe2cd4c-8384-4636-8359-5847f42b9711 [Accessed 17/03/2026]

Stancu, V.S., Haugaard, P.H. and Lähteenmäki, L.L. (2016) Determinants of Consumer Food Waste Behaviour: Two Routes to Food Waste. [online]., p. 1. [Accessed 18 March 2026].

Di Vaio, A.D.V., Boccia, F.B., Landriani, L.L. and Palladino, R.P. (2020) Artificial Intelligence in the Agri-food System: Rethinking Sustainable Business Models in the Covid-19 Scenario. [online]., p. 3. [Accessed 18 March 2026].

Rodrigues, M.R., Miguéis, V.M., Freitas, S.F., Machado, T.M. and , (2024) . Machine Learning Models For Short-term Demand Forecasting in Food Catering Services: A Solution to Reduce Food Waste [online]., p. 4. [Accessed 18 March 2026].

Song, PS.(2025)ML Journey. *Time Series Forecasting with Prophet vs ARIMA*[blog].22 September. Available from Time Series Forecasting with Prophet vs ARIMA - ML Journey [Accessed 19/03/2026]

Kaur, T.K. (2025) Traditional Prediction Models: Prophet & Arima. [online]. [Accessed 19 March 2026].

Facebook (n.d.) Prophet: Forecasting at scale. Available from: https://facebook.github.io/prophet/ [Accessed 19 March 2026].

Yasar, K.Y. (2024) What Is Web Application (Web Apps) and Its Benefits?. [online]. [Accessed 22 March 2026].

Amazon Web Services (n.d.) *What's the Difference Between Web Apps, Native Apps, and Hybrid Apps?* Available from: https://aws.amazon.com/compare/the-difference-between-web-apps-native-apps-and-hybrid-apps/ [Accessed 22 March 2026].

Al-shamaileh, O.A. and Sutcliffe, A.S. (2023) Why People Choose Apps: An Evaluation of the Ecology and User Experience of Mobile Applications. [online]. 170 [Accessed 22 March 2026].

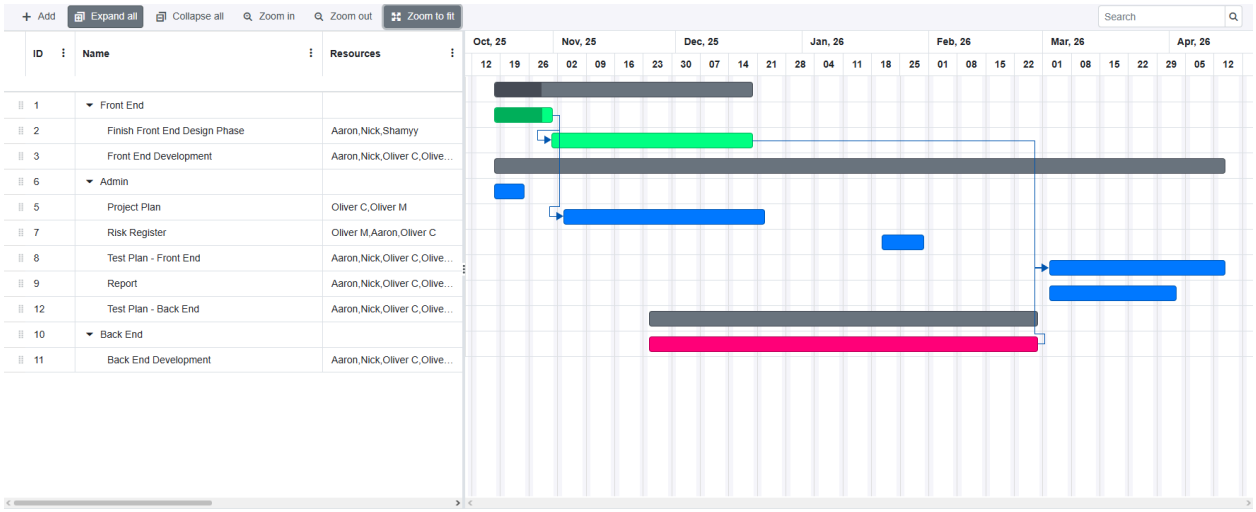
React (n.d.) *React documentation*. Available from: https://react.dev [Accessed 22 March 2026].
Tailwind Labs (n.d.) *Tailwind CSS documentation*. Available from: https://tailwindcss.com [Accessed 22 March 2026].

Pallets Projects (n.d.) *Flask documentation*. Available from: https://flask.palletsprojects.com/en/stable/ [Accessed 22 March 2026].

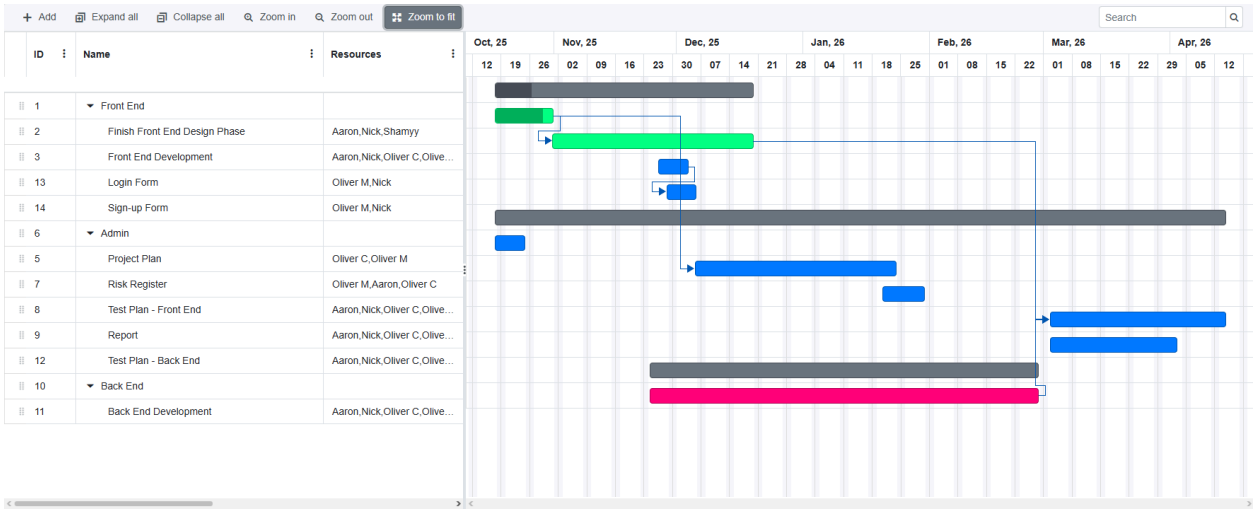
SQLite (n.d.) *SQLite documentation*. Available from: https://sqlite.org [Accessed 22 March 2026].

Appendices

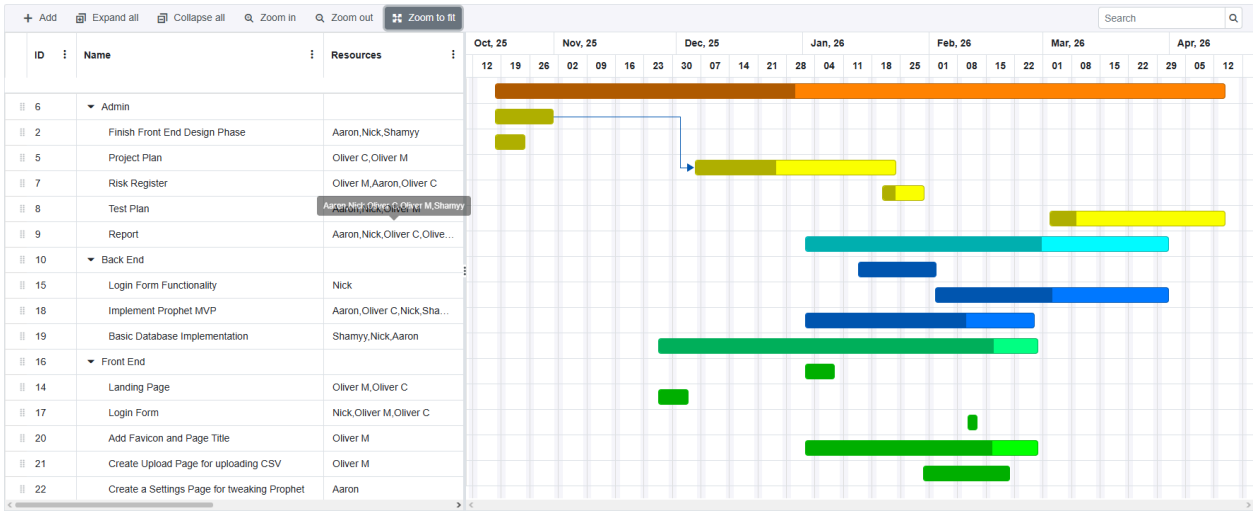
Gantt Chart - 17/10/2025 Creation:



Gantt Chart - 24/11/2025 Update:



Gantt Chart - 20/02/2026 Update:



Peer feedback:

1. Very Realistic Objective following a good structure and clear. should focus more on project orientated objectives rather than project management
2. Functional and non functional definitions incorrect and would need rework structure of powerpoint however current information is strong
3. project plan was realistic and had achievable goals
4. simple and detailed with correct information on their test plan and following correct terminology, specified by the lecturers
5. very nice website with easy access and following real world examples for user friendly use.
6. powerpoint structure needs to be reworked to follow a better flow within the presentation. however all information used was strong and valid

Brief tutor feedback for improvement:

Good progress on the implementation. Docker and GitHub are used well for project management. It would be even better to combine them with a Gantt chart for project planning and management, and to highlight this a bit in the final presentation and report. The functional and non-functional requirements could be phrased more clearly. The overall logic and flow of the slides should be improved.